

Providence Marketplace Guide

System Architecture & Feature Reference Guide

1. Project Architecture Overview

The Providence E-Commerce Marketplace website is a modern, high-performance customer-to-customer (C2C) platform constructed using a decoupled architecture. It provides standard users the flexibility to act as both buyers and sellers seamlessly.

A. Frontend Architecture (React & Vite)

- **Framework & Build Engine:** Built using React JS, compiled and bundled via Vite for optimized fast-load times and Hot Module Replacement.
- **Routing & Navigation:** Utilizes react-router-dom for client-side nested layouts and router guards protecting Admin Panel resources.
- **State Management:** Implements the React Context API (AppContext.jsx) as a centralized state store for shopping carts, user sessions, and credentials caching.
- **Design Language:** Styled with Vanilla CSS (index.css) using a dark-theme glassmorphism design system to present a premium interface.

B. Backend Architecture (Node.js, Express & MongoDB)

- **Application Server:** Powered by Node.js running an Express.js server mapping API routers and checking input validations.
- **Database Engine:** Uses MongoDB Atlas cloud database managed via the Mongoose ODM library for schema modeling, index mapping, and document retrieval.
- **Security Middleware:** Utilizes custom middlewares verifying JWT (JSON Web Tokens) to decrypt credentials, populate requests (req.user), and enforce admin-only boundaries.

2. Core Functions & Modular Components

A. User Authentication & Profile Protection

Allows secure client signup, secure login verification, and profile visual options.

- **Security Lockout:** Administrator profile details are protected by backend validation and frontend states, presenting a read-only screen to prevent modifications.
- **Key Modules:** bcryptjs (safe password salting and hashing) and jsonwebtoken (generating JWT session tokens with a 30-day expiry).

B. Product Catalog & Image File Uploads

Allows standard clients to list products for sale by inputting standard details (title, description, price, town location) along with uploading local image files.

- **Multer storage integration:** The backend integrates Multer to parse incoming multipart/form-data requests, rename images safely with unique suffixes, and store files inside the /uploads folder.
- **Static Serving:** Express.static middleware hosts the files statically, letting the frontend view images directly from localhost URLs.
- **Key Modules:** multer (multipart processing) and FormData Web API (multipart formatting).

C. Product Search, Filtering & Browsing

Allows buyers to dynamically search for listings based on matching words, specific categories, or location criteria.

- **Key Modules:** Mongoose RegExp matching queries (supporting case-insensitive searches).

D. Shopping Cart & Secure Checkout

Provides shopping cart quantities updates, order total logic, delivery shipping forms, and validation checks for debit/credit cards.

- **Key Modules:** Mongoose Schema models saving transaction orders to MongoDB.

E. Administrative Panel & Metrics Reports

Gathers global e-commerce metrics (revenue calculations, listing totals, active accounts registries, suspend/unsuspend toggles, and user orders logs).

- **Key Modules:** Mongoose queries (aggregating Paid orders and summing total amount properties).

3. Technical Dependency Registry

The following table details the key modules/packages installed in the project codebase, listing their application layer and primary responsibilities:

Package	Layer	Description / Primary Purpose
express	Backend	HTTP web server framework managing routing, requests, and
mongoose	Backend	Object Data Modeling (ODM) managing database schemas and
multer	Backend	Parses incoming multipart form uploads to save image files onto server
bcryptjs	Backend	Secure cryptographically hashed encryption shielding client passwords.
jsonwebtoken	Backend	Generates JWT authentication structures for user credentials
cors	Backend	Cross-Origin Resource Sharing control facilitating requests between
dotenv	Backend	Loads environmental configurations (.env) securely on startup.
react	Frontend	Builds reactive user interfaces and handles hooks states.
react-router-dom	Frontend	Maps URL paths to client layouts and implements dashboard path
lucide-react	Frontend	Vector icons kit rendering status indicators, metrics cards, and lock